

1. Backpropagation through time (BPTT) in RNNs

After the RNN outputs the prediction vector $h(t)$, we compute the prediction error $E(t)$ and use the Back Propagation Through time algorithm to compute the gradient

$$(1) \quad \frac{\partial E}{\partial W} = \sum_{t=1}^T \frac{\partial E_t}{\partial W}.$$

The gradient is used to update the model parameters by:

$$(2) \quad W \leftarrow W - \alpha \frac{\partial E}{\partial W}.$$

And we continue the learning process using the Gradient Descent algorithm.

Say we have learning task that includes T time steps, the gradient of the error on the k time step is given by:

$$(3) \quad \frac{\partial E_k}{\partial W} = \frac{\partial E}{\partial h_k} \frac{\partial h_k}{\partial c_k} \dots \frac{\partial c_2}{\partial c_1} \frac{\partial c_1}{\partial W} = \frac{\partial E_k}{\partial h_k} \frac{\partial h_k}{\partial c_k} \left(\prod_{t=2}^k \frac{\partial c_t}{\partial c_{t-1}} \right) \frac{\partial c_1}{\partial W}$$

Notice that since $W = [W_{hh}, W_{hx}]$, $c(t)$ can be written as:

$$(4) \quad c(t) = \tanh(W_{hh}c_{t-1} + W_{hx}x_t).$$

Compute the derivative of $c(t)$ and get:

$$(5) \quad \frac{\partial c_t}{\partial c_{t-1}} = \tanh'(W_{hh}c_{t-1} + W_{hx}x_t) \frac{\partial}{\partial c_{t-1}} [W_{hh}c_{t-1} + W_{hx}x_t] = \tanh'(W_{hh}c_{t-1} + W_{hx}x_t) W_{hh}$$

Plug Eq. (5) into Eq. (3) and get our backpropagated gradient

$$(6) \quad \frac{\partial E_k}{\partial W} = \frac{\partial E_k}{\partial h_k} \frac{\partial h_k}{\partial c_k} \left(\prod_{t=2}^k \tanh'(W_{hh}c_{t-1} + W_{hx}x_t) W_{hh} \right) \frac{\partial c_1}{\partial W}$$

The last expression tends to vanish when k is large, this is due to the derivative of the tanh activation function which is smaller than 1.

The product of derivatives can also explode if the weights W_{hh} are large enough to overpower the smaller tanh derivative, this is known as the exploding gradient problem.

2. Long Short-Term Memory

An LSTM network has an input vector $[h(t-1), x(t)]$ at time step t . The network cell state is denoted by $c(t)$. The output vectors passed through the network between consecutive time steps $t, t+1$ are denoted by $h(t)$.

An LSTM network has three gates that update and control the cell states, these are the forget gate, input gate and output gate. The gates use hyperbolic tangent and sigmoid activation functions. The forget gate controls what information in the cell state to forget, given new information than entered the network.

The forget gate's output is given by:

$$(7) \quad f_t = \sigma(W_{fh}h_{t-1} + W_{fx}x_t).$$

The input gate controls what new information will be encoded into the cell state, given the new input information.

The input gate's output has the form:

$$(8) \quad \tanh(W_{hh}h_{t-1} + W_{hx}x_t) \otimes \sigma(W_{ih}h_{t-1} + W_{ix}x_t),$$

where we can define

$$(9) \quad \tilde{c}_t = \tanh(W_{hh}h_{t-1} + W_{hx}x_t)$$

and

$$(10) \quad i_t = \sigma(W_{ih}h_{t-1} + W_{ix}x_t).$$

The output gate controls what information encoded in the cell state is sent to the network as input in the following time step, this is done via the output vector $h(t)$. The output gate's activations are given by:

$$(11) \quad o_t = \sigma(W_{oh}h_{t-1} + W_{ox}x_t)$$

and the cell's output vector is given by:

$$(12) \quad h_t = o_t \otimes \tanh(c_t)$$

The long term dependencies and relations are encoded in the cell state vectors and it's the cell state derivative that can prevent the LSTM gradients from vanishing. The LSTM cell state has the form:

$$(13) \quad c_t = c_{t-1} \otimes f_t + \tilde{c}_t \otimes i_t.$$

2.1. Backpropagation through time in LSTMs. As in the RNN model, our LSTM network outputs a prediction vector $h(k)$ on the k -th time step. The knowledge encoded in the state vectors $c(t)$ captures long-term dependencies and relations in the sequential data.

The length of the data sequences can be hundreds and even thousands of time steps, making it extremely difficult to learn using a basic RNN. We compute the gradient used to update the network parameters, the computation is done over T time steps.

As in RNNs, the error term gradient is given by the following sum of T gradients:

$$(14) \quad \frac{\partial E}{\partial W} = \sum_{t=1}^T \frac{\partial E_t}{\partial W}.$$

The gradient of the error for some time step k has the form:

$$(15) \quad \frac{\partial E_k}{\partial W} = \frac{\partial E}{\partial h_k} \frac{\partial h_k}{\partial c_k} \dots \frac{\partial c_2}{\partial c_1} \frac{\partial c_1}{\partial W} = \frac{\partial E_k}{\partial h_k} \frac{\partial h_k}{\partial c_k} \left(\prod_{t=2}^k \frac{\partial c_t}{\partial c_{t-1}} \right) \frac{\partial c_1}{\partial W}$$

As we have seen, the following product causes the gradients to vanish:

$$(16) \quad \prod_{t=2}^k \frac{\partial c_t}{\partial c_{t-1}}$$

In an LTSM, the state vector $c(t)$, has the form:

$$(17) \quad c_t = c_{t-1} \otimes f_t + \tilde{c}_t \otimes i_t.$$

Notice that the state vector $c(t)$ is a function of the following elements, which should be taken into account when computing the derivative during backpropagation:

$$(18) \quad c_{t-1}, \quad f_t, \quad \tilde{c}_t, \quad i_t$$

Compute the derivative of Eq. (17) and get:

$$(19) \quad \frac{\partial c_t}{\partial c_{t-1}} = \frac{\partial}{\partial c_{t-1}}[c_{t-1} \otimes f_t + \tilde{c}_t \otimes i_t] = \frac{\partial}{\partial c_{t-1}}[c_{t-1} \otimes f_t] + \frac{\partial}{\partial c_{t-1}}[\tilde{c}_t \otimes i_t]$$

$$(20) \quad = \frac{\partial f_t}{\partial c_{t-1}} c_{t-1} + f_t + \frac{\partial i_t}{\partial c_{t-1}} \tilde{c}_t + \frac{\partial \tilde{c}_t}{\partial c_{t-1}} i_t,$$

where the four derivative terms are respectively denoted as A_t, B_t, C_t and D_t . The LSTM states gradient can be written as

$$(21) \quad \frac{\partial E_k}{\partial W} = \frac{\partial E_k}{\partial h_k} \frac{\partial h_k}{\partial c_k} \left(\prod_{t=2}^k [A_t + B_t + C_t + D_t] \right) \frac{\partial c_1}{\partial W}$$

Notice that the gradient contains the forget gate's vector of activations, which allows the network to better control the gradients values, at each time step, using suitable parameter updates of the forget gate. The presence of the forget gate's activations allows the LSTM to decide, at each time step, that certain information should not be forgotten and to update the model's parameters accordingly.

Say that for some time step $k < T$, we have that: $\sum_{i=1}^k \frac{\partial E_i}{\partial W} \rightarrow 0$. Then for the gradient not to vanish, we can find a suitable parameter update of the forget gate at time step $k+1$ such that $\frac{\partial E_{k+1}}{\partial W} \not\rightarrow 0$.

It is the presence of the forget gate's vector of activations in the gradient term along with additive structure which allows the LSTM to find such a parameter update at any time step, and this yields: $\sum_{i=1}^{k+1} \frac{\partial E_i}{\partial W} \not\rightarrow 0$, and the gradient doesn't vanish.

Another important property to notice is that the cell state gradient is an additive function made up from four elements denoted $A(t), B(t), C(t), D(t)$. This additive property enables better balancing of gradient values during backpropagation. The LSTM updates and balances the values of the four components making it more likely the additive expression does not vanish. This additive property is different from the RNN case where the gradient contained a single element inside the product.

In LSTMs, however, the presence of the forget gate, along with the additive property of the cell state gradients, enables the network to update the parameter in such a way that the different sub gradients do not necessarily agree and behave in a similar manner, making it less likely that all of the T gradients will vanish, or in other words, the series of functions does not converge to zero, and our gradients do not vanish.

Summing up, we have seen that RNNs suffer from vanishing gradients and caused by long series of multiplications of small values, diminishing the gradients and causing the learning process to become degenerate. In a analogous way, RNNs suffer from exploding gradients affected from large gradient values and hampering the learning process.

LSTMs solve the problem using a unique additive gradient structure that includes direct access to the forget gate's activations, enabling the network to encourage desired behaviour from the error gradient using frequent gates update on every time step of the learning process.

3. Sequence-to-Sequence

The seq2seq model was born in the field of language modeling. Broadly speaking, it aims to transform an input sequence (source) to a new one (target) and both sequences can be of arbitrary lengths, e.g., machine translation between multiple languages.

The seq2seq model (see Figure 1) normally has an encoder-decoder architecture, composed of:

- An **encoder** processes the input sequence and compresses the information into a context vector (also known as sentence embedding or “thought” vector) of a fixed length. This representation is expected to be a good summary of the meaning of the whole source sequence.
- A **decoder** is initialized with the context vector to emit the transformed output. The early work only used the last state of the encoder network as the decoder initial state.

Both the encoder and decoder are recurrent neural networks, i.e. using LSTM or GRU units.

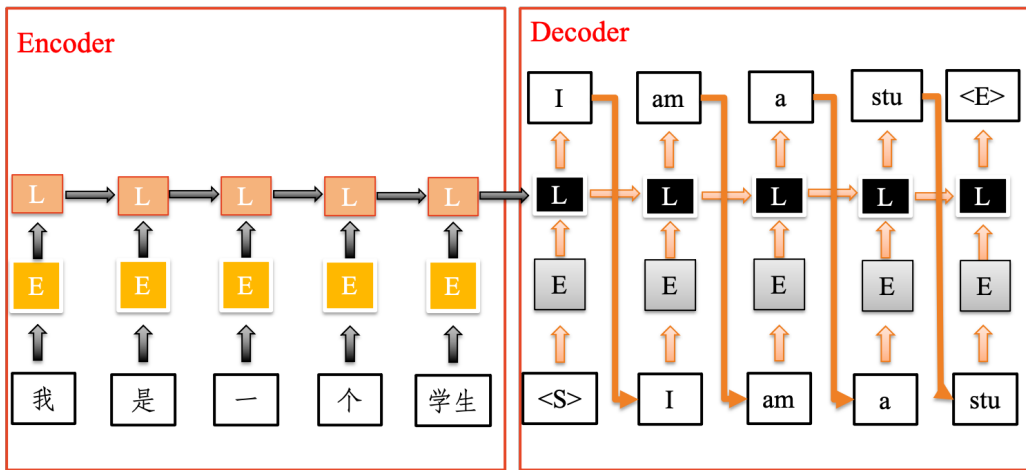


FIGURE 1. A Seq2Seq model for machine translation.

A critical and apparent disadvantage of this fixed-length context vector design is incapability of remembering long sentences. Often it has forgotten the first part once it completes processing the whole input. The attention mechanism was born to resolve this problem, as shown in Figure 2.

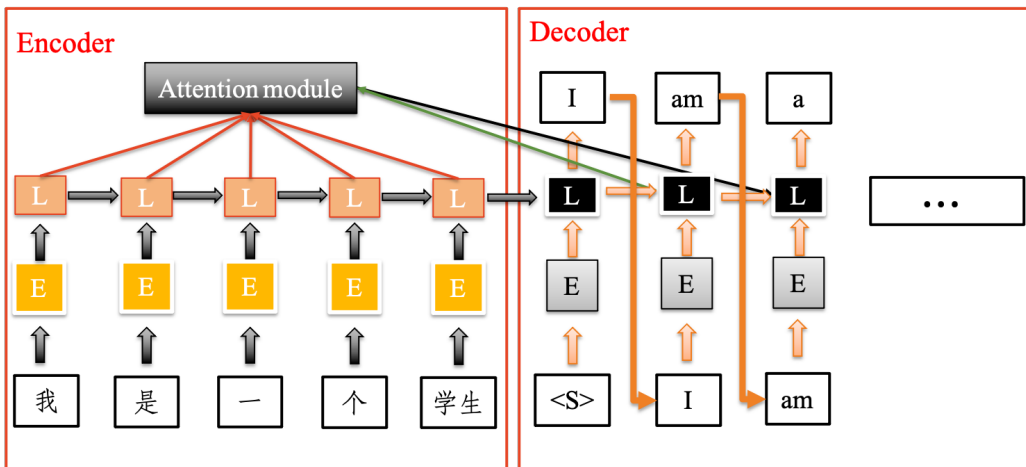


FIGURE 2. A Seq2Seq with an attention model for machine translation.

4. Attention

Now let's define the attention mechanism introduced in NMT in a scientific way. Say, we have a source sequence x of length n and try to output a target sequence y of length m

$$(22) \quad x = [x_1, x_2, \dots, x_n]$$

$$(23) \quad y = [y_1, y_2, \dots, y_m]$$

The encoder is a bidirectional RNN with a hidden state h_i . The decoder network has hidden state $s_t = f(s_{t-1}, y_{t-1}, c_t)$ for the output word at position t , $t = 1, \dots, m$, where the context vector c_t is a sum of hidden states of the input sequence, weighted by alignment scores:

$$(24) \quad c_t = \sum_{i=1}^n \alpha_{t,i} h_i$$

$$(25) \quad \alpha_{t,i} = \frac{\exp(\text{score}(s_{t-1}, h_i))}{\sum_{i'=1}^n \exp(\text{score}(s_{t-1}, h_{i'}))}$$

The alignment model assigns a score $\alpha_{t,i}$ to the pair of input at position i and output at position t , (y_t, x_i) , based on how well they match. The set of $\{\alpha_{t,i}\}$ are weights defining how much of each source hidden state should be considered for each output. The alignment score α is parametrized by a feed-forward network with a single hidden layer and this network is jointly trained with other parts of the model. The score function is therefore in the following form, given that $\tanh$ is used as the non-linear activation function:

$$(26) \quad \text{score}(s_t, h_i) = v_a^T \tanh(W_a[s_t; h_i]),$$

where both v_a and W_a are weight matrices to be learned in the alignment model. The matrix of alignment scores (see Figure 3) is a nice byproduct to explicitly show the correlation between source and target words.

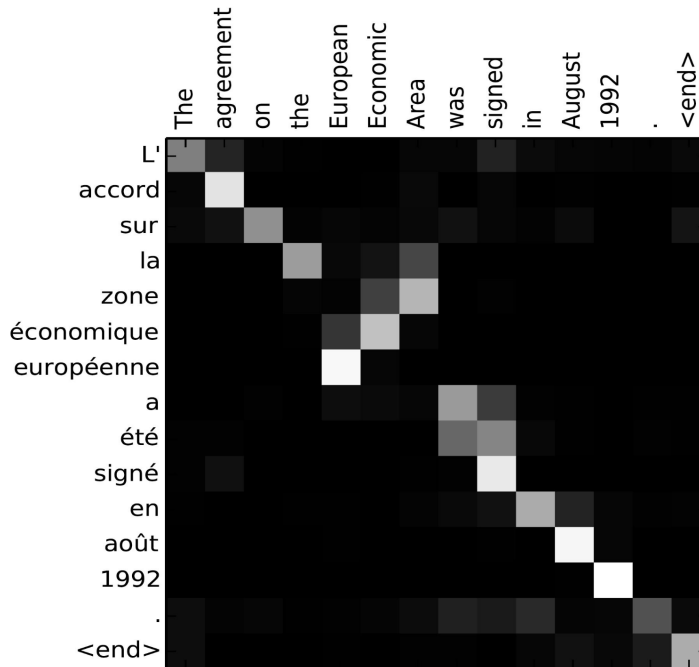


FIGURE 3. Alignment matrix of “L’accord sur l’Espace économique européen a été signé en août 1992” (French) and its English translation “The agreement on the European Economic Area was signed in August 1992”.

With the help of the attention, the dependencies between source and target sequences are not restricted by the in-between distance anymore! Given the big improvement by attention in machine translation, it soon got extended into the computer vision field and people started exploring various other forms of attention mechanisms.

5. Self-Attention

Self-attention, also known as intra-attention, is an attention mechanism relating different positions of a single sequence in order to compute a representation of the same sequence. It has been shown to be very useful in machine reading, abstractive summarization, or image description generation.

The “transformer” model is entirely built on the self-attention mechanisms without using sequence-aligned recurrent architecture.

5.1. Key, Value and Query. The major component in the transformer is the unit of multi-head self-attention mechanism. The transformer views the encoded representation of the input as a set of key-value pairs, (K, V) , both of dimension n (input sequence length); in the context of NMT, both the keys and values are the encoder hidden states. In the decoder, the previous output is compressed into a query (Q of dimension m) and the next output is produced by mapping this query and the set of keys and values.

The transformer adopts the scaled dot-product attention: the output is a weighted sum of the values, where the weight assigned to each value is determined by the dot-product of the query with all the keys:

$$(27) \quad \text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{n}}\right)V$$

The key/value/query concepts come from retrieval systems. For example, when you type a query to search for some video on Youtube, the search engine will map your query against a set of keys (video title, description, etc.) associated with candidate videos in the database, then present you the best matched videos (values).

Eq. (27) can also be understood as soft addressing, as shown in Figure 4. If each element is stored in a (K, V) in the form of a sequence, then the attention addressing is done by calculating the Q and K similarity. Q and K reflect the calculated similarity value is taken out of the V importance, i.e. the weight, and then summing the weighted values to get attention.

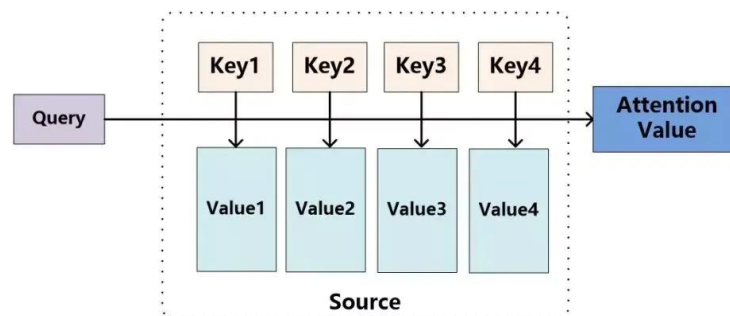


FIGURE 4. Soft addressing.

Remember that Q_i and K_i were different projections of the tokens into a n dimensional space, i.e, $Q_i = W^q a_i$ and $K_i = W^k a_i$. Therefore, we can think about the dot product of those projections as a measure of similarity between tokens projections. For every token projected through Q_i the dot product with the projections through K_i measures the similarity between those tokens. The

self-attention is exactly to relate different tokens!

Rather than only computing the attention once, the multi-head mechanism runs through the scaled dot-product attention multiple times in parallel. The independent attention outputs are simply concatenated and linearly transformed into the expected dimensions. According to the paper, “multi-head attention allows the model to jointly attend to information from different representation subspaces at different positions. With a single attention head, averaging inhibits this.”